Лабораторная работа №1
Анализ предметной области и требования

Дисциплина: Технологии разработки программного обеспечения
Студент: Черноусов Олег Андреевич
Группа: еозИСИП - 303
Тема: Система учёта заявок

1. Цель работы

Научиться переводить исходную задачу в формализованные требования к программной системе.

2. Ход работы

2.1. Создание структуры проекта.

Создана папка проекта `Chernousov_Oleg_eozISIP303_RequestSystem` со структурой: `docs` (документация), `src` (исходный код), `tests` (тесты). Проект инициализирован в Git и выложен в публичный репозиторий GitHub: [ссылка на репозиторий].

2.2. Описание предметной области.

Система учёта заявок предназначена для регистрации, обработки и контроля обращений пользователей организации. Без системы заявки принимаются по телефону, почте и в мессенджерах, теряются и дублируются, а руководитель не видит реальную загрузку исполнителей и сроки выполнения. Система централизует обращения в одном месте, назначает исполнителей, отслеживает статусы и формирует историю работ.

2.3. Роли пользователей.

В системе выделены 4 роли: администратор (управление пользователями, категориями, просмотр отчётов), оператор (регистрация заявок, назначение исполнителя), исполнитель (выполнение назначенных заявок, изменение статуса), заявитель (создание заявок, просмотр своих заявок, закрытие выполненных). Подробные права каждой роли приведены в техническом задании (`docs/TZ.md`, раздел 2).

2.4. Требования.

Сформулировано 12 функциональных требований (регистрация заявки, присвоение номера, статусы «Новая»/«В работе»/«Выполнена»/«Закрыта», назначение исполнителя, категории, комментарии, фильтрация, отчёты) и 5 нефункциональных требований по категориям: удобство, надёжность, производительность, безопасность, сопровождаемость. Полный текст — в `docs/TZ.md`.

2.5. Сущности предметной области.
Выделены сущности: Пользователь, Заявка, Статус, Категория, Комментарий, История статусов.

2.6. Диаграмма Use Case
Построена диаграмма вариантов использования (файл `docs/usecase.png`). На ней отражены 4 актёра и основные варианты использования: создание заявки, просмотр своих заявок, закрытие заявки, назначение исполнителя, взятие в работу, выполнение заявки, формирование отчёта, управление категориями и пользователями.

2.7. Критерии приёмки.
Сформулированы критерии готовности системы: регистрация заявки с уникальным номером, назначение исполнителя, смена статусов, наличие 4 категорий, разграничение доступа (заявитель видит только свои заявки), фильтрация, отчёт, сохранение данных в БД, наличие Git-репозитория и README.

2.8. Работа с нейросетью
Для анализа предметной области использовалась нейросеть DEEPSEEK-v3. Были сформулированы два промпта: свободный и структурированный (с указанием роли, контекста, формата и ограничений). Структурированный промпт дал более полезный результат: полный, проверяемый список требований без общих фраз. Из ответа ИИ взяты роли, функциональные и нефункциональные требования, сущности и критерии приёмки. Проверка показала, что часть требований была двусмысленной (например, «отправлять уведомления» — не указано, кому и какие), поэтому они были переформулированы в проверяемые («отображать заявителю статус его заявки»). Также самостоятельно добавлена история изменения статусов, которой в ответе ИИ не было.

3. AI-журнал

Промпт 1 (свободный):
«У меня есть система учёта заявок. Помоги составить требования: роли пользователей, функциональные и нефункциональные требования, сущности и критерии приёмки.»

Промпт 2 (структурированный):
«Ты — системный аналитик с опытом 10 лет. Контекст: разрабатывается система учёта заявок для организации, пользователи — администратор, оператор, исполнитель, заявитель. Задача: составь техническое задание. Формат ответа: 1) назначение (3–5 предложений); 2) таблица ролей и прав (4 роли); 3) не менее 10 функциональных требований в формате «Система должна…»; 4) не

менее 5 нефункциональных требований по категориям (удобство, надёжность, производительность, безопасность, сопровождаемость); 5) не менее 5 сущностей предметной области; 6) не менее 5 сценариев Use Case; 7) критерии приёмки. Ограничения: без воды, без выдуманных технологий, требования проверяемые. Критерий качества: по каждому требованию понятно, как его проверить.»

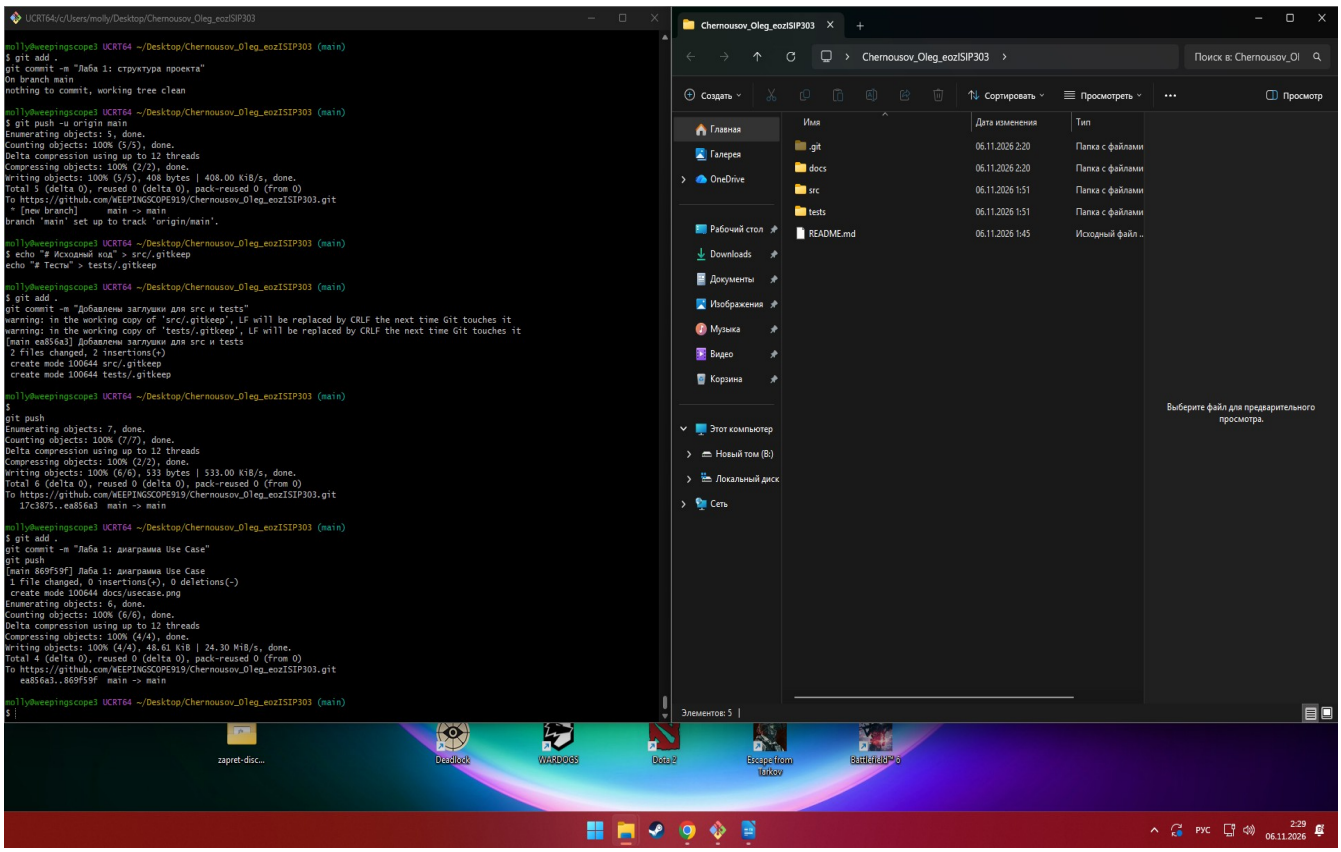
Сравнение: структурированный промпт дал более полезный результат, потому что задал роль, контекст, формат и ограничения — ответ получился полным и проверяемым. Свободный промпт дал поверхностный список, который пришлось додумывать.

Вывод

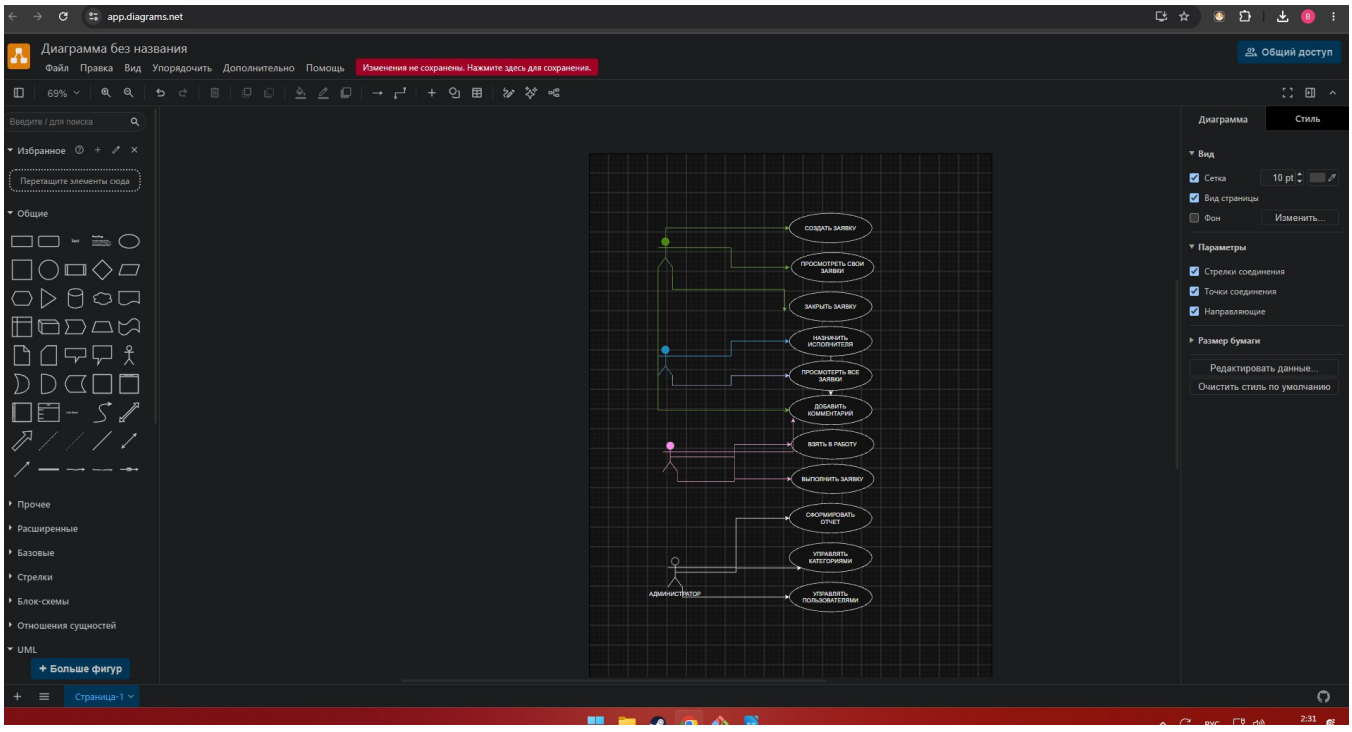
В ходе работы освоен перевод исходной задачи в формализованные требования: выделены роли, функциональные и нефункциональные требования, сущности, сценарии использования и критерии приёмки. Построена диаграмма Use Case, соответствующая тексту технического задания. Проект размещён в Git-репозитории. Нейросеть использовалась как инструмент анализа: её ответ проверен, двусмысленные требования исправлены, часть требований доработана самостоятельно.

Ниже скриншоты/примеры выполнения заданий:

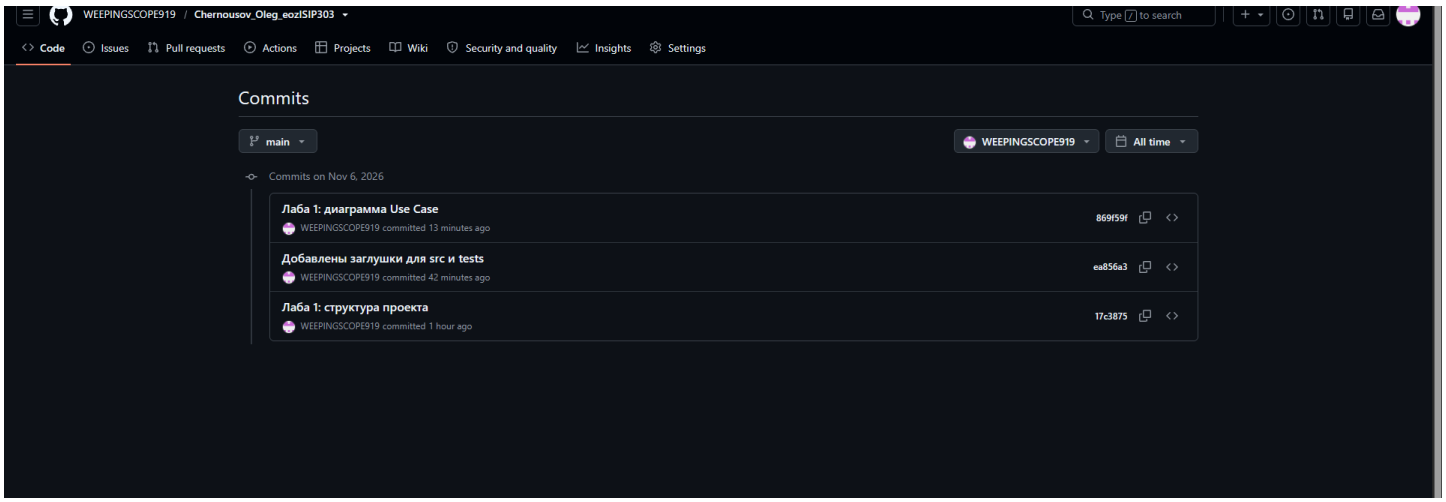
1.Работа с репозиторием гитхаба с помощю git



2. Создание диаграммы с помощью https://app.diagrams.net



3.Обновленные коммиты на гите


WEEPINGSCOPE919 / Chernousov_Oleg_eozISIP303
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
Commits
main
WEEPINGSCOPE919
All time
Commits on Nov 6, 2026
Лаба 1: диаграмма Use Case
WEEPINGSCOPE919 committed 13 minutes ago
869f59f
Добавлены заглушки для src и tests
WEEPINGSCOPE919 committed 42 minutes ago
ea856a3
Лаба 1: структура проекта
WEEPINGSCOPE919 committed 1 hour ago
17c3875